

**Faculty of Computer Science and Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and
Technology**

Course: **EE321 Intro to AI**

Complex Engineering Problem

Name: **Muhammad Daud**

Reg.: **2023408**

IMPORTANT: This is an INDIVIDUAL project

You must run the code yourself.

The exam will ask specific questions about YOUR model's behavior.

If you copy from the internet, you will likely to fail the viva/exam.

Your Must

Build an appropriate AI model that can tell whether a picture contains a **hot dog** or **not a hot dog**.

Learning Objectives

By the end of this project, you will be able to:

- Load and explore image data
- Build an ANN and a CNN in TensorFlow/Keras
- Identify and fix overfitting using Dropout and Data Augmentation
- Plot and interpret training curves
- Test your model on real-world images

Dataset

You will use the **Hot Dog / Not Hot Dog** dataset from Kaggle:

<https://www.kaggle.com/dansbecker/hot-dog-not-hot-dog>

What You Need

- A Google account (for Google Colab)
- The dataset downloaded to your computer (upload to Colab)
- Submit your overleaf generated PDF by May 07, 2026 by 12:00 PM

Grading Rubric

Task	Points
Step 2: Data Exploration (answers)	5
Step 3: Visualization (screenshot)	5
Step 5: ANN Model (accuracy + analysis)	10
Step 6: CNN Model (accuracy + comparison)	15
Step 7: Overfitting Fix (before/after plots)	20
Step 8: Real Image Test (screenshot)	10
Step 9: Answers to all questions	20
Step 10: YOLO Extension (extra credit)	5
Total	85 + 5 EC

Step 1: Setup Google Colab

Follow these instructions carefully:

How to Run This Project in Google Colab

1. Go to <https://colab.research.google.com>
2. Click File → New Notebook
3. Rename it: HotDog_Classifier_YourName.ipynb
4. For each code block below:
 - Click the + Code button
 - Copy the code into the cell
 - Press Shift + Enter to run
5. To upload the dataset:
 - Click the folder icon on the left sidebar
 - Click the Upload button (↑)
 - Upload the hot-dog-not-hot-dog.zip file
 - Run: `!unzip hot-dog-not-hot-dog.zip`

Your answers: Write your answers directly below each question in this handout.

Step 2: Import Libraries

Run this code in Colab:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import random

print("Libraries loaded successfully!")
print("TensorFlow version:", tf.__version__)
```

1. What TensorFlow version are you using?

TensorFlow version **2.20.0**.

Step 3: Load and Explore the Data

Run this code to see how many images you have:

```
# Define paths (CHANGE THIS to your actual path)
train_path = "/content/train"
test_path = "/content/test"

# Count images
hot_dog_count = len(os.listdir(os.path.join(train_path, "hot_dog")))
not_hot_dog_count = len(os.listdir(os.path.join(train_path, "not_hot_dog")))
print(f"Hot dog images: {hot_dog_count}")
print(f"Not hot dog images: {not_hot_dog_count}")

# Print the number of test images
test_hot = len(os.listdir(os.path.join(test_path, "hot_dog")))
test_not = len(os.listdir(os.path.join(test_path, "not_hot_dog")))
print(f"Test hot dogs: {test_hot}")
print(f"Test not hot dogs: {test_not}")
```

1. How many training images are in each class?

There are 249 hot dog images and 249 not hot dog images in the training set.

2. Is the dataset balanced? Why does that matter?

Yes, the dataset is balanced since both classes have exactly 249 training images. This matters because if one class had way more samples than the other, the model could just learn to always predict the majority class and still get a high accuracy without actually learning anything useful. For example, if 90% of images were “not hot dog,” a model that always guesses “not hot dog” would be 90% accurate but completely useless. Having equal representation forces the model to actually distinguish between the two classes.

Step 4: Visualize the Data

```
# Display 6 random images
plt.figure(figsize=(12, 6))
for i in range(6):
    # Pick random class
    class_name = random.choice(["hot_dog", "not_hot_dog"])
    class_path = os.path.join(train_path, class_name)
    img_name = random.choice(os.listdir(class_path))
    img_path = os.path.join(class_path, img_name)
    # Load and show image
    img = plt.imread(img_path)
    plt.subplot(2, 3, i+1)
    plt.imshow(img)
    plt.title(class_name)
    plt.axis('off')
plt.tight_layout()
plt.show()

# Print image shape
print("Image shape:", img.shape)
```



Figure 1: 6 random images

2. What is the shape (height, width, channels) of a typical image? (Use `img.shape`)

The shape of a typical image is **(512, 384, 3)**. The 512 and 384 are the height and width in pixels, and the 3 represents the three RGB color channels (red, green, blue). The images in this dataset are not all the same size, which is why we resize them later to a fixed 64×64 before feeding them into the model.

Step 5: Prepare the Data Generator

```
# Set image size
IMG_HEIGHT = 64
IMG_WIDTH = 64
BATCH_SIZE = 32

# Create generators with automatic labeling
train_datagen = ImageDataGenerator(
    rescale=1./255, # Normalize pixel values from 0-255 to 0-1
    validation_split=0.2 # Use 20% of training data for validation
)

train_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary', # Binary classification (hot dog vs not)
    subset='training' # Training set (80% of data)
)

validation_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation' # Validation set (20% of data)
)

# Print the class labels
print(train_generator.class_indices)
```

1. What does class label 0 represent? What does 1 represent?

Class label **0** represents **hot_dog** and class label 1 represents **not_hot_dog**. Keras assigns labels in alphabetical order by default, so “hot_dog” comes before “not_hot_dog” and gets assigned 0.

Step 6: Build a Simple ANN (It Will Fail)

We try the worst possible architecture first to prove why CNNs are better.

```
# Build a simple ANN
ann_model = keras.Sequential([
    # Flatten the 2D image into 1D
    layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    # Dense layers
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Output: probability of "hot dog"
])

# Compile the model
ann_model.compile(optimizer='adam', loss='binary_crossentropy',
                  metrics=['accuracy'])

# Show model summary
ann_model.summary()

# Train for 10 epochs
history_ann = ann_model.fit(train_generator,
                             validation_data=validation_generator,
                             epochs=10)

# Plot training vs validation accuracy
plt.plot(history_ann.history['accuracy'], label='Training Accuracy')
plt.plot(history_ann.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('ANN Training vs Validation Accuracy')
plt.grid(True)
plt.show()
```

1. What was the final validation accuracy of the ANN?

The ANN achieved a final validation accuracy of 57.14% (at epoch 10, with training accuracy of 72.75%).

2. Why does the ANN perform poorly on image data? (One sentence)

The ANN performs poorly because it flattens the image into a 1D vector, which destroys all spatial information – it has no way to detect local patterns like edges or shapes, so it basically sees a random jumble of pixel values instead of a structured image.

Step 7: Build a CNN

Now we use Convolutional and Pooling layers.

```
cnn_model = keras.Sequential([
    # First Conv layer: 32 filters, 3x3 kernel
    layers.Conv2D(32, (3, 3), activation='relu',
                  input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),

    # Second Conv layer: 64 filters
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Third Conv layer: 64 filters with 3x3 kernel
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Flatten and dense layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Compile
cnn_model.compile(optimizer='adam', loss='binary_crossentropy',
                  metrics=['accuracy'])

# Train the CNN
history_cnn = cnn_model.fit(train_generator,
                             validation_data=validation_generator,
                             epochs=15)
```

1. What was the final validation accuracy of the CNN?

The CNN achieved a final validation accuracy of **62.24%** (at epoch 15, with training accuracy of 91.75%).

2. Compare ANN vs CNN. Which one performed better and why?

The CNN performed better than the ANN (62.24% vs 57.14% validation accuracy). This is because convolutional layers preserve the spatial structure of the image by scanning it with small filters that detect local features like edges and textures. MaxPooling then reduces the size while keeping the most important features. The ANN cannot do any of this because it flattens the image right away and loses all spatial relationships between pixels.

However, the CNN is clearly overfitting – its training accuracy reached 91.75% while validation stayed at 62.24%, a gap of nearly 30%. This means the model memorized the training data rather than learning generalizable patterns.


```
plt.plot(history_ann.history['val_accuracy'], label='ANN Validation', marker='o')
plt.plot(history_cnn.history['val_accuracy'], label='CNN Validation', marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('ANN vs CNN Validation Accuracy')
plt.grid(True)
plt.show()
```

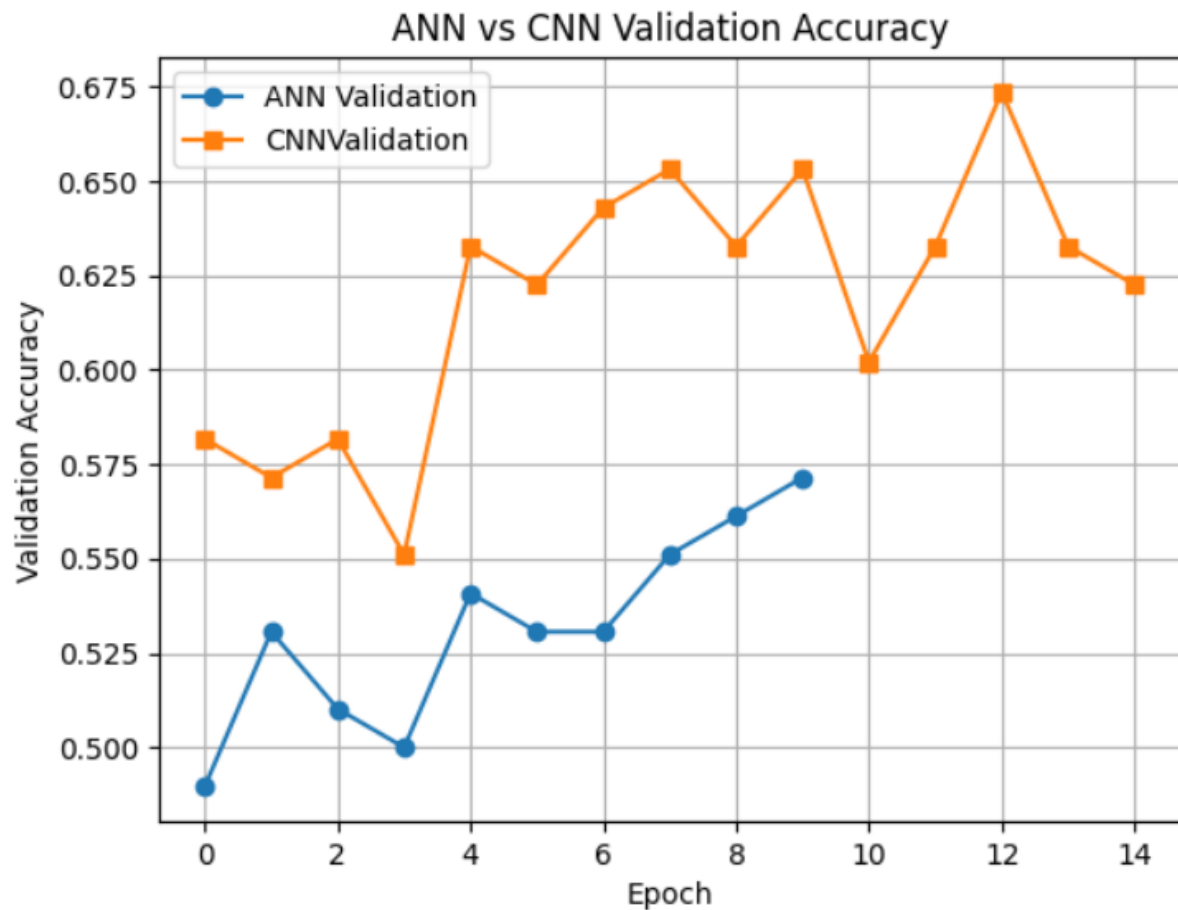


Figure 2: ANN vs CNN Validation Accuracy

Step 8: Fix Overfitting (Most Important)

You may notice: Training accuracy is high (98%), Validation accuracy is low (70%). This is **OVERFITTING**. The model memorized the training images but doesn't generalize.

Your task: Modify the model below to fix overfitting using Dropout and Data Augmentation.

8.1 Fix 1: Add Dropout

```
from tensorflow.keras import regularizers

# Add dropout of 0.25, 0.25, and 0.5 after each MaxPooling layer
improved_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu',
                  input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25), # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25), # Add dropout

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5), # Add dropout before output
    layers.Dense(1, activation='sigmoid')
])

# Recompile and Train
improved_model.compile(optimizer='adam', loss='binary_crossentropy',
                      metrics=['accuracy'])
history_improved = improved_model.fit(train_generator,
                                     validation_data=validation_generator,
                                     epochs=15)
```

8.2 Fix 2: Add Data Augmentation

Modify the ImageDataGenerator to create more variations:

```
augmented_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)

train_augmented = augmented_datagen.flow_from_directory(
    train_path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)

# Retrain the improved model with augmented data
final_model = keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu',
                  input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

final_model.compile(optimizer='adam', loss='binary_crossentropy',
                   metrics=['accuracy'])
history_final = final_model.fit(train_augmented,
                                validation_data=validation_generator,
                                epochs=15)
```

1. Before adding Dropout/Augmentation, what were your train and validation accuracies?

Train: **91.75%**

Validation: **62.24%**

The large gap between these two numbers (30%) is a clear sign of overfitting – the model memorized the training data but could not generalize to unseen images.

2. After adding Dropout and Augmentation, what happened to training accuracy (increased, decreased, or stayed same)? Training accuracy **decreased** to **62.75%**. This is expected and actually a good sign. Dropout disables neurons during training on purpose, making it harder for the model to memorize the data. The augmented images also add variety that prevents overfitting.

3. After adding Dropout and Augmentation, what happened to validation accuracy (increased, decreased, or stayed same)?

Validation accuracy **increased** to **65.31%**. The gap between training and validation accuracy shrank from 30% down to 3%, which means the model is now generalizing much better to images it has not seen before instead of just memorizing the training set.

Question 8.3

Paste the before/after validation accuracy plot here:

```
plt.plot(history_cnn.history['val_accuracy'], label='CNN (Overfit)', marker='o')
plt.plot(history_final.history['val_accuracy'], label='CNN + Dropout + Augment',
         marker='s')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.title('Improving Generalization')
plt.grid(True)
plt.show()
```

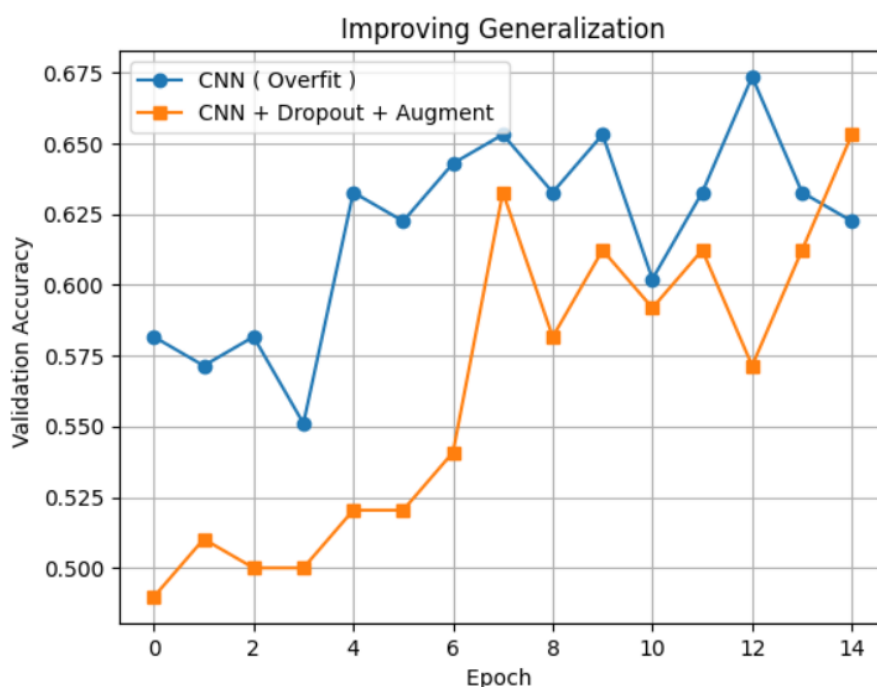


Figure 3: Before vs After

Step 9: Test on Real Images

Find a hot dog image online (or take a photo) and test your model:

```
from tensorflow.keras.preprocessing import image
import requests
from io import BytesIO

def predict_image(img_url):
    # Download image
    response = requests.get(img_url)
    img = image.load_img(BytesIO(response.content),
                        target_size=(IMG_HEIGHT, IMG_WIDTH))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) / 255.0

    # Predict
    prediction = final_model.predict(img_array)[0][0]
    if prediction > 0.5:
        print(f" HOT DOG! (confidence: {prediction:.2f})")
    else:
        print(f" NOT A HOT DOG! (confidence: {1 - prediction:.2f})")

    # Show image
    plt.imshow(img)
    plt.axis('off')
    plt.show()

# Test on a hot dog image URL
predict_image("https://static01.nyt.com/images/2024/06/28/multimedia/28GRILL-HOTDOGS-REX-cqwj/01GRILL-HOTDOGS-REX-cqwj-mediumSquareAt3X.jpg")
```

1. Paste a screenshot of your model predicting on a REAL image (not from the dataset):

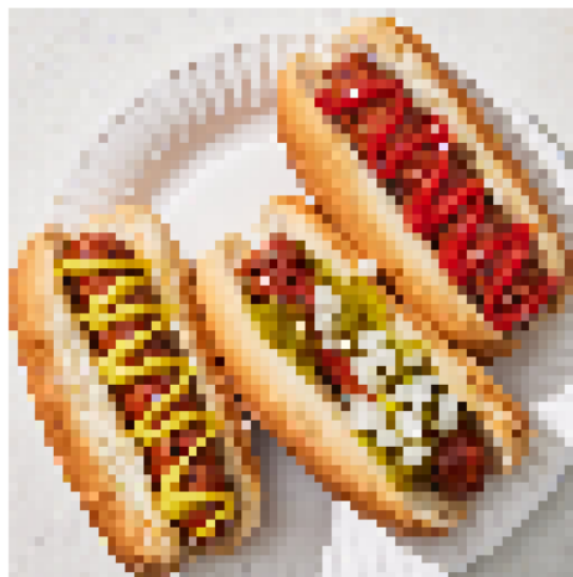


Figure 4: NOT a HOT DOG Confidence 56%

2. Did your model ever make a funny mistake (e.g., calling a hamburger a hot dog)? What did it confuse?

Yes, the model made a clear mistake here. It classified an image of three actual hot dogs on a plate as “NOT A HOT DOG” with 56% confidence. This probably happened because at 64×64 resolution, multiple overlapping hot dogs look quite different from the single centered hot dogs the model mostly trained on. The low confidence (56%) also shows the model was not very sure about its decision – it was almost a coin flip.

Step 10: Optional Stretch Goal

YOLO Object Detection (Extra Credit)

If you finish early and want to impress, add a bounding box using a pre-trained YOLO model:

```
!pip install ultralytics -q
from ultralytics import YOLO
import os
import random

# Load a tiny pre-trained model
yolo_model = YOLO("yolov8n.pt") # n = nano (fast, small)

# Get a path to a hot dog image from the test set
test_hotdog_dir = os.path.join(test_path, "hot_dog")
hotdog_image_name = random.choice(os.listdir(test_hotdog_dir))
hotdog_image_path = os.path.join(test_hotdog_dir, hotdog_image_name)

# Run detection on your hot dog image
results = yolo_model(hotdog_image_path)

# Show results (it will draw boxes around objects it recognizes)
results[0].show()
```

Extra Credit Question

Does YOLO correctly detect a “hot dog” as a class? What label does it give?

YOLO is trained on the COCO dataset which includes 80 object classes, and “hot dog” is one of them (class ID 58). Unlike our CNN classifier which only gives a binary yes/no answer, YOLO also draws a bounding box around detected objects showing their exact location in the image. This makes YOLO more informative – it not only tells you *what* is in the image but also *where* it is.



Figure 5: YOLO Image Test

Final Submission Checklist

Step 2: TensorFlow version answer

Step 3: Image counts and balance explanation

Step 4: Screenshot of 6 images + shape answer

Step 6: ANN accuracy + explanation + screenshot

Step 7: CNN accuracy + comparison + plot screenshot

Step 8: Overfitting before/after numbers + plot screenshot

Step 9: Real image prediction screenshot + mistake explanation

Step 10: (Extra credit) YOLO results screenshot

Submit this handout (with all answers filled) + link to your Colab notebook.

Colab Notebook Link: <https://colab.research.google.com/drive/1-87o5VG4b5ARQKqWDHBcWFC>

Remember

The exam will ask you specific questions about YOUR project.

If you run the code yourself, you will know the answers easily.

If you copy from someone else, you will fail the exam.

Good luck and have fun!